CSE 102 Assignment 3 Report

Assignment Objective:

The goal of this assignment was to develop a Virtual ATM Simulator in C that allows basic banking transactions such as checking balance, depositing money, and withdrawing money. The application must use scalar variables only, conditional statements (if, else, switch), and file I/O. No arrays, loops, structs, or pointers were allowed. Each operation must be implemented through functions.

Implemented Functions (From the Code):

- **get_balance();**

```c
double get_balance(){ //Reads balance from file

    FILE *dosya;
    dosya = fopen("account.txt", "r");

    if(dosya == NULL){

        dosya = fopen("account.txt", "w");
        fprintf(dosya, "100.00");
        fclose(dosya);
        return 100.00;
    }


    double balance;

    fscanf(dosya, "%lf", &balance);
    fclose(dosya);

    return balance;
    }
```

Reads the account balance from the file. If the file does not exist, it creates it with an initial balance of 100.00.

- **update_balance(double new_balance)**

```c
void update_balance(double new_balance){ //Updates balance in the file

    FILE *dosya;
    dosya = fopen("account.txt", "w");

    fprintf(dosya, "%2.f", new_balance);
    fclose(dosya);
    }
```

Updates the balance in the file by overwriting the old one.

- **check_balance();**

```c
void check_balance(){ //Displays current balance

   double balance = get_balance();
   printf("Current balance: %2.f\n", balance);

}
```

Displays the current balance on the screen using the value returned by get_balance().

- **deposit_money();**

```c
void deposit_money() { //Allows user to deposit money

    double amount;

    printf("Enter amount to deposit: ");
    scanf("%lf", &amount);

       if (amount <= 0) {
        printf("Invalid amount. Amount must be positive.\n");
        return;
    }

    double balance = get_balance();
    balance += amount;
    update_balance(balance);
    printf("Deposit successful! New balance: %.2f TRY\n", balance);
}
```

Handles user input, ensures positive values, and updates the file with the new balance.

- **withdraw_money();**

```c
void withdraw_money(){ //Allows user to withdraw money

    double amount;
    printf("Enter amount to withdraw: ");
    scanf("%lf", &amount);

    if (amount <= 0) {
        printf("Invalid amount. Amount must be positive.\n");
        return;
    }

    double balance = get_balance();

    if (amount > balance) {
        printf("Insufficient funds! Your balance is only %.2f TRY.\n", balance);
        return;
    }

    balance -= amount;
    update_balance(balance);
    printf("Withdraw successful! New balance: %.2f TRY\n", balance);
}
```

Handles withdrawal requests, including insufficient fund checks, and updates the file accordingly.

- menu();

```
void menu(){ //Displays the ATM Menu

    int option;

    printf("----- Virtual ATM -----\n");
    printf("1. Check Balance\n");
    printf("2. Deposit Money\n");
    printf("3. Withdraw Money\n");
    printf("4. Exit\n");
    printf("Choose an option: \n");
    scanf("%d", &option);

switch(option) {

    case 1:
    check_balance();
    break;

    case 2:
    deposit_money();
    break;

    case 3:
    withdraw_money();
    break;

    case 4:
    break;

    default:
    printf("Invalid option.\n");
}
}
```

Displays the transaction menu, receives user input, and calls the corresponding functions.

Outputs

```
----- Virtual ATM -----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option:
1
Current balance: 300

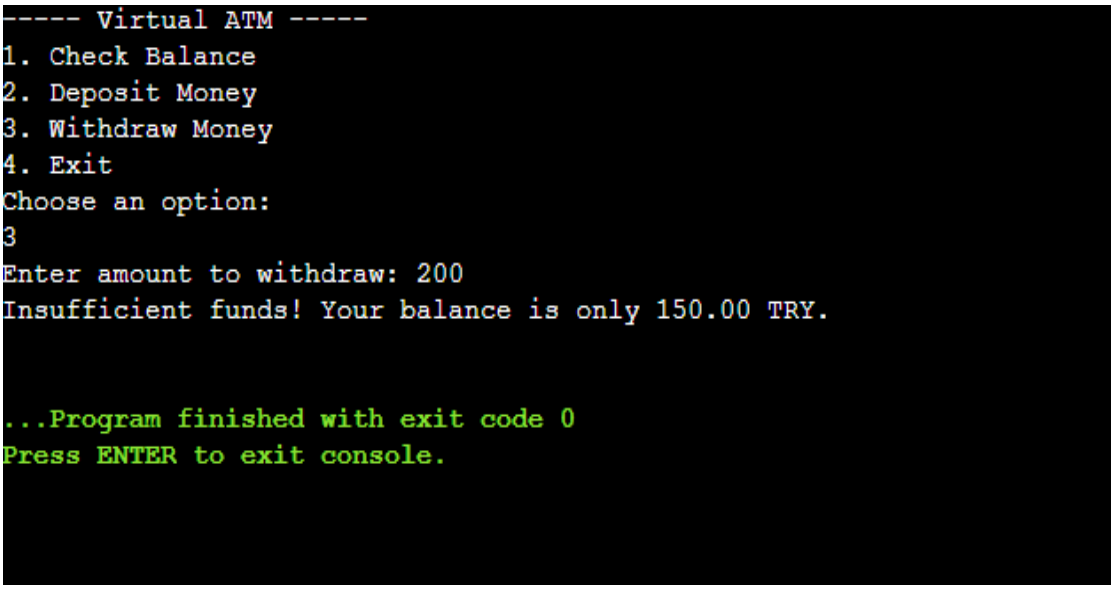

...Program finished with exit code 0
Press ENTER to exit console.
```

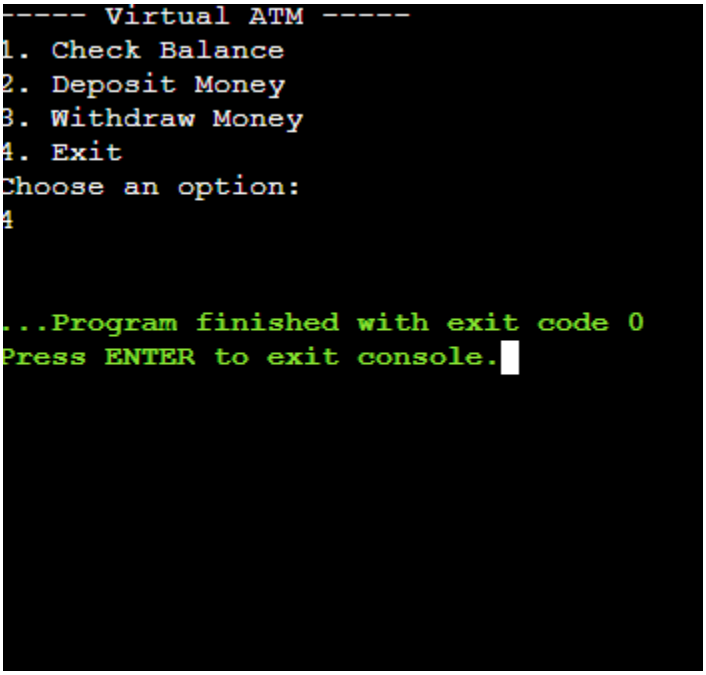
```
----- Virtual ATM -----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option:
2
Enter amount to deposit: 50
Deposit successful! New balance: 350.00 TRY


...Program finished with exit code 0
Press ENTER to exit console.
```

```
----- Virtual ATM -----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Choose an option:
3
Enter amount to withdraw: 200
Withdraw successful! New balance: 150.00 TRY


...Program finished with exit code 0
Press ENTER to exit console.
```





Conclusion

The program fulfills all technical and functional requirements. It uses clean modular code and adheres strictly to the given constraints. The output format is appropriate and error handling (negative inputs, insufficient funds) is also implemented.